



**Instituto Superior  
de Engenharia**

Politécnico de Coimbra

Programação Web– 2025/2026

Licenciatura em Engenharia Informática

João Oliveira Nº 2023144718

Guilherme Moreira Nº 2023144718

## Índice

|        |                                                              |    |
|--------|--------------------------------------------------------------|----|
| 1.     | Introdução .....                                             | 3  |
| 1.1.   | Objetivo .....                                               | 3  |
| 1.2.   | Objetivos Específicos .....                                  | 6  |
| 2.     | Arquitetura da Solução .....                                 | 7  |
| 3.     | Completo do Trabalho .....                                   | 8  |
| 3.1.   | Funcionalidades do Frontend (MyMediaWeb / MyMediaMAUI) ..... | 8  |
| 3.2.   | Funcionalidades do Backoffice (GestaoLoja) .....             | 9  |
| 3.3.   | Funcionalidades da API RESTful .....                         | 10 |
| 4.     | Modelo Entidade Relacionamento .....                         | 11 |
| 4.1.   | Diagrama .....                                               | 11 |
| 4.2.   | Entidades .....                                              | 11 |
| 4.2.1. | <b>ApplicationUser</b> .....                                 | 11 |
| 4.2.2. | Cliente .....                                                | 12 |
| 4.2.3. | Fornecedor .....                                             | 12 |
| 4.2.4. | Produtos .....                                               | 12 |
| 4.2.5. | Categorias e Subcategorias .....                             | 13 |
| 4.2.6. | Vendas e LinhasVenda .....                                   | 13 |
| 5.     | Regras de Negócio .....                                      | 15 |
| 5.1.   | Regras Gerais .....                                          | 15 |
| 5.2.   | Regras Utilizadores .....                                    | 15 |
| 5.3.   | Regras de Produtos .....                                     | 15 |
| 5.4.   | Regras de Vendas .....                                       | 16 |
| 5.5.   | Regras de Categorias e Subcategorias .....                   | 16 |
| 5.6.   | Regras de Stock .....                                        | 16 |
| 6.     | Credenciais de Acesso .....                                  | 17 |
| 6.1.   | Aplicação de Gestão (GestãoLoja) .....                       | 17 |
| 6.2.   | Aplicação Frontend (MyMediaWeb / MyMediaMAUI) .....          | 17 |
| 7.     | Conclusão .....                                              | 18 |

# 1. Introdução

## 1.1. Objetivo

O presente trabalho prático tem como objetivo principal o desenho, desenvolvimento e implementação de uma solução de software distribuída para comércio eletrónico de artigos multimédia, denominada MyMEDIA. A solução foi concebida para operar num modelo de negócio de intermediação, onde a plataforma serve de ponte entre fornecedores (proprietários dos artigos) e clientes finais, sendo gerida administrativamente por uma entidade central.

Os objetivos específicos do projeto, em conformidade com o enunciado proposto, foram atingidos através das seguintes implementações:

**Desenvolvimento Multi-Plataforma:** Criação de uma aplicação cliente ("Frontend") capaz de operar em ambiente Web e Mobile (Android/iOS) partilhando uma base de código única, recorrendo à tecnologia .NET MAUI Blazor Hybrid.

**Arquitetura Orientada a Serviços:** Implementação de uma API RESTful robusta que centraliza a lógica de acesso a dados e serve as aplicações cliente de forma segura e desacoplada.

**Backoffice de Gestão:** Desenvolvimento de uma aplicação Web dedicada ("Gestão de Loja") para administração integral do sistema, permitindo a gestão de utilizadores, a validação de produtos inseridos por fornecedores e o controlo de vendas.

**Segurança e Controlo de Acessos:** Implementação de mecanismos de autenticação e autorização transversais, utilizando ASP.NET Core Identity e JWT (JSON Web Tokens), garantindo a distinção correta de permissões entre os diversos perfis de utilizador (Visitante, Cliente, Fornecedor, Funcionário e Administrador).

**Persistência de Dados:** Modelação e implementação de uma base de dados relacional SQL Server, partilhada entre o sistema de gestão e a API, garantindo a integridade e consistência da informação.

A concretização destes objetivos permitiu criar um ecossistema aplicacional funcional, escalável e alinhado com as tecnologias mais recentes do ecossistema .NET 8.

O trabalho desenvolvido contempla a totalidade das funcionalidades especificadas no enunciado do projeto, constituindo uma prova de conceito funcional e demonstrativa de uma aplicação com potencial de utilização real no contexto do comércio eletrónico.

As principais áreas funcionais implementadas são as seguintes:

### **Sistema de Autenticação e Autorização**

- Implementação do ASP.NET Core Identity com suporte para quatro perfis de utilizador distintos: Administrador, Funcionário, Cliente e Fornecedor;
- Autenticação baseada em cookies na aplicação de gestão (GestaoLoja);
- Autenticação baseada em JWT na comunicação entre o frontend e a API RESTful;
- Sistema de registo com aprovação manual — novos utilizadores (Clientes e Fornecedores) iniciam com estado "Pendente" até serem aprovados por um Administrador ou Funcionário.

### **Gestão de Produtos**

- Operações CRUD completas (Criar, Ler, Atualizar, Eliminar) sobre produtos;
- Sistema de estados para controlo do ciclo de vida dos produtos (Pendente, Ativo, Inativo);
- Mecanismo de aprovação de produtos inseridos ou editados por Fornecedores;
- Definição de percentagem da empresa aquando da aprovação;
- Cálculo automático do preço final de venda;
- Gestão de stock com atualização automática aquando da confirmação de vendas.

### **Catálogo e Navegação**

- Organização hierárquica em Categorias e Subcategorias;
- Interface com frisos deslizantes para navegação por categorias e subcategorias;
- Funcionalidade de produto em destaque com seleção aleatória;
- Sistema de pesquisa por nome de produto, categoria ou subcategoria;
- Filtragem de produtos por categoria e subcategoria selecionadas.

### **Carrinho de Compras e Checkout**

- Adição e remoção de produtos do carrinho;
- Alteração de quantidades dos itens no carrinho;
- Cálculo automático de subtotais e total da encomenda;
- Processo de checkout com recolha de dados de entrega;
- Simulação de múltiplos métodos de pagamento (Cartão de Crédito/Débito, MB WAY, Multibanco);
- Persistência do carrinho durante a sessão do utilizador.

### **Gestão de Vendas**

- Fluxo completo de estados das vendas: Pendente → Confirmada → Expedida;
- Possibilidade de rejeição de vendas pendentes;
- Atualização automática de stocks aquando da confirmação;
- Dashboard com indicadores de vendas (pendentes, confirmadas, expedidas, total mensal);
- Histórico de compras para Clientes;
- Histórico de vendas para Fornecedores.

### **Gestão de Utilizadores**

- Aprovação de registos de Clientes e Fornecedores;
- Ativação e desativação de contas de utilizadores;
- Gestão de Funcionários (exclusiva para Administradores);
- Proteção contra auto-desativação de contas de Administrador.

### **Interface de Utilizador**

- Design responsivo adaptado a diferentes dimensões de ecrã;
- Interface única partilhada entre aplicação Web e aplicação móvel;
- Feedback visual através de spinners de carregamento e mensagens de estado.

## 1.2. Objetivos Específicos

Frontend (MyMediaWeb e MyMediaMAUI):

- Visualização de produtos com filtros por categoria e subcategoria
- Produto em destaque (aleatório)
- Pesquisa de produtos
- Carrinho de compras
- Registo de Clientes e Fornecedores (estado Pendente)
- Login com autenticação JWT
- Histórico de compras (Cliente)
- Gestão de produtos próprios (Fornecedor)
- Histórico de vendas (Fornecedor)

Backoffice (GestaoLoja):

- Dashboard com estatísticas de vendas
- CRUD completo de Categorias, Subcategorias, Produtos e Modos de Entrega
- Gestão de Clientes e Fornecedores (aprovar/ativar/inativar)
- Gestão de Vendas (confirmar/rejeitar/expedir)
- Definição de percentagem de lucro por produto
- Controlo de permissões por perfil (Administrador vs Funcionário)

API RESTful:

- Endpoints públicos e protegidos
- Autenticação JWT
- Documentação Swagger
- Padrão Repository

## 2. Arquitetura da Solução

**MyMediaWeb (Blazor Web App)** Aplicação Web Blazor Server para acesso via browser. Consome a API RESTful para todas as operações de dados e utiliza os componentes das RCLs para a interface de utilizador.

**MyMediaMAUI (MAUI Blazor Hybrid)** Aplicação híbrida para dispositivos móveis e desktop (Windows, Android, iOS, macOS). Partilha o mesmo código de interface através das RCLs, diferenciando-se apenas no método de armazenamento local (Preferences em vez de localStorage).

**RESTfulAPIWeb (ASP.NET Core Web API)** API RESTful que serve como intermediário entre as aplicações frontend e a base de dados. Implementa autenticação JWT, documentação Swagger e o padrão Repository para acesso a dados.

**GestaoLoja (Blazor Web App)** Aplicação de backoffice para administração da loja. Acede diretamente à base de dados através do Entity Framework Core, sem passar pela API. Utiliza autenticação por cookies do Identity.

### Razor Class Libraries (RCLs)

- **RCLAPI:** Serviços de comunicação com a API (ApiService, CarrinhoService)
- **RCLProdutos:** Componentes de visualização de produtos (Home, Produto, Pesquisa, MeusProdutos)
- **RCLCompras:** Componentes de compras (Carrinho, Checkout, MinhasCompras, MinhasVendasFornecedor)
- **RCLComum:** Componentes comuns partilhados

### 3. Completude do Trabalho

#### 3.1. Funcionalidades do Frontend (MyMediaWeb / MyMediaMAUI)

| Funcionalidade                                 | Estado   |
|------------------------------------------------|----------|
| Visualizar produtos disponíveis                | Completo |
| Filtrar produtos por categoria                 | Completo |
| Filtrar produtos por subcategoria              | Completo |
| Pesquisar produtos por nome                    | Completo |
| Ver detalhes de um produto                     | Completo |
| Ver produto em destaque (aleatório)            | Completo |
| Adicionar produtos ao carrinho                 | Completo |
| Registrar-se como Cliente (estado Pendente)    | Completo |
| Registrar-se como Fornecedor (estado Pendente) | Completo |
| Login com autenticação JWT                     | Completo |
| Finalizar compra (checkout)                    | Completo |
| Simular pagamento                              | Completo |
| Consultar histórico de compras (Cliente)       | Completo |
| Ver estado das encomendas (Cliente)            | Completo |
| Inserir novos produtos (Fornecedor)            | Completo |
| Editar produtos próprios (Fornecedor)          | Completo |
| Suspender/Reativar produtos (Fornecedor)       | Completo |
| Consultar histórico de vendas (Fornecedor)     | Completo |
|                                                |          |

### 3.2. Funcionalidades do Backoffice (GestaoLoja)

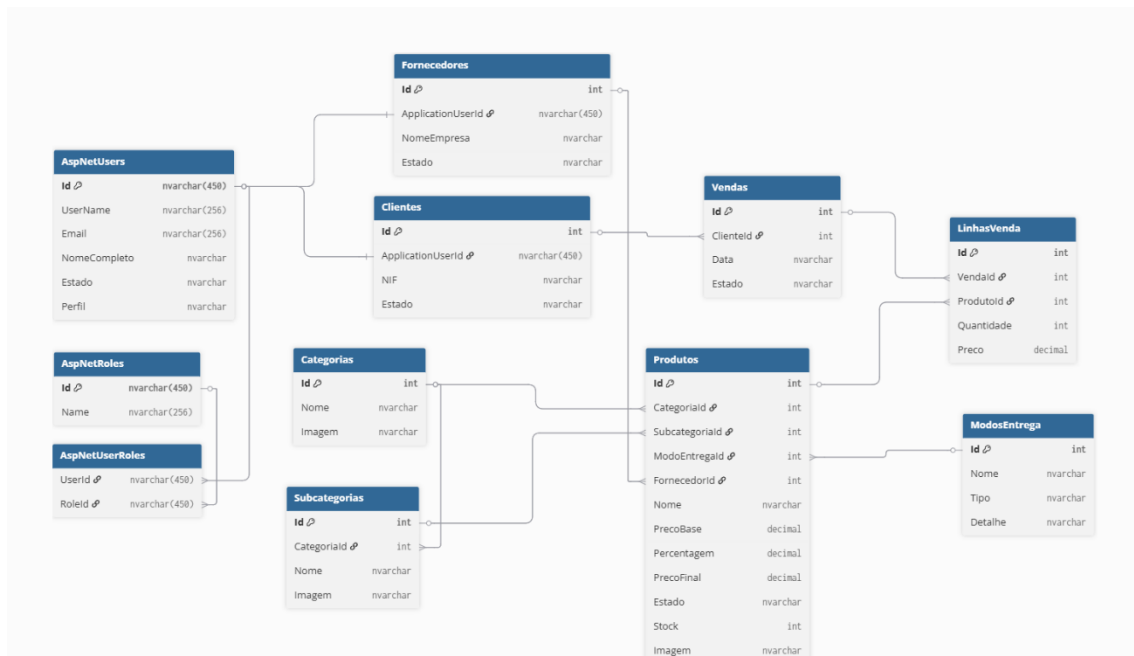
| Funcionalidade                        | Estado   |
|---------------------------------------|----------|
| Dashboard com estatísticas de vendas  | Completo |
| Contadores de vendas por estado       | Completo |
| Total de vendas do mês                | Completo |
| Listar categorias                     | Completo |
| Criar categoria                       | Completo |
| Editar categoria                      | Completo |
| Apagar categoria (se sem produtos)    | Completo |
| Listar subcategorias                  | Completo |
| Criar subcategoria                    | Completo |
| Editar subcategoria                   | Completo |
| Apagar subcategoria (se sem produtos) | Completo |
| Listar produtos com filtros           | Completo |
| Ordenação de produtos                 | Completo |
| Paginação de produtos                 | Completo |
| Criar produto                         | Completo |
| Editar produto                        | Completo |
| Apagar produto (se sem vendas)        | Completo |
| Aprovar produtos pendentes            | Completo |
| Ativar/Inativar produtos              | Completo |
| Definir percentagem de lucro          | Completo |
| Listar modos de entrega               | Completo |
| Criar modo de entrega                 | Completo |
| Editar modo de entrega                | Completo |
| Apagar modo de entrega                | Completo |
| Listar vendas com filtros por estado  | Completo |
| Confirmar vendas                      | Completo |
| Rejeitar vendas                       | Completo |
| Expedir vendas                        | Completo |
| Atualização automática de stocks      | Completo |
| Listar clientes                       | Completo |
| Aprovar/Ativar clientes               | Completo |
| Inativar clientes                     | Completo |
| Listar fornecedores                   | Completo |
| Aprovar fornecedores                  | Completo |
| Inativar fornecedores                 | Completo |
|                                       |          |

### 3.3. Funcionalidades da API RESTful

| Endpoint                                    | Descrição                   | Estado   |
|---------------------------------------------|-----------------------------|----------|
| GET /api/categorias                         | Listar categorias           | Completo |
| GET /api/subcategorias                      | Listar subcategorias        | Completo |
| GET /api/subcategorias/categoria/{id}       | Subcategorias por categoria | Completo |
| GET /api/produtos                           | Listar produtos ativos      | Completo |
| GET /api/produtos/{id}                      | Detalhes de um produto      | Completo |
| GET /api/produtos/pesquisa                  | Pesquisar produtos          | Completo |
| GET /api/produtos/categoria/{id}            | Produtos por categoria      | Completo |
| GET /api/produtos/subcategoria/{id}         | Produtos por subcategoria   | Completo |
| GET /api/produtos/destaque                  | Produto em destaque         | Completo |
| GET /api/modosentrega                       | Listar modos de entrega     | Completo |
| POST /api/auth/login                        | Login com JWT               | Completo |
| POST /api/auth/register-cliente             | Registo de cliente          | Completo |
| POST /api/auth/register-fornecedor          | Registo de fornecedor       | Completo |
| POST /api/vendas                            | Criar venda                 | Completo |
| GET /api/vendas/minhas                      | Minhas compras              | Completo |
| POST /api/vendas/{id}/pagar                 | Simular pagamento           | Completo |
| GET /api/fornecedor/produtos                | Meus produtos               | Completo |
| POST /api/fornecedor/produtos               | Criar produto               | Completo |
| PUT /api/fornecedor/produtos/{id}           | Editar produto              | Completo |
| PUT /api/fornecedor/produtos/{id}/suspender | Suspender produto           | Completo |
| PUT /api/fornecedor/produtos/{id}/reativar  | Reativar produto            | Completo |
| DELETE /api/fornecedor/produtos/{id}        | Apagar produto              | Completo |
| GET /api/fornecedor/produtos/vendas         | Minhas vendas               | Completo |
| POST /api/imagens/produto                   | Upload de imagem            | Completo |
|                                             |                             |          |

## 4. Modelo Entidade Relacionamento

### 4.1. Diagrama



### 4.2. Entidades

#### 4.2.1. ApplicationUser

Esta entidade estende a classe base do ASP.NET Core Identity, adicionando campos específicos da aplicação:

```
// GestaoLoja/Data/ApplicationUser.cs
public class ApplicationUser : IdentityUser
{
    public string NomeCompleto { get; set; } = string.Empty;

    // Estado: Ativo / Inativo / Pendente
    public string Estado { get; set; } = "Ativo";

    // Perfil: Administrador / Funcionário / Cliente / Fornecedor
    public string Perfil { get; set; } = "Cliente";

    // Relações opcionais
    public Cliente? Cliente { get; set; }
    public Fornecedor? Fornecedor { get; set; }
}
```

### 4.2.2. Cliente

Dados específicos dos utilizadores com perfil Cliente:

```
// GestaoLoja/Entities/Cliente.cs
public class Cliente
{
    public int Id { get; set; }

    // Ligação ao utilizador Identity
    public string ApplicationUserId { get; set; } = default!;
    public ApplicationUser? ApplicationUser { get; set; }

    // Dados próprios do cliente
    public string NIF { get; set; } = "";
    public string Estado { get; set; } = "Pendente"; // Pendente, Ativo,
Inativo

    // Relação com vendas
    public ICollection<Vendas>? Vendas { get; set; }
}
```

### 4.2.3. Fornecedor

Dados específicos dos utilizadores com perfil Fornecedor:

```
// GestaoLoja/Entities/Fornecedor.cs
public class Fornecedor
{
    public int Id { get; set; }

    // Ligação ao utilizador
    public string ApplicationUserId { get; set; } = default!;
    public ApplicationUser? ApplicationUser { get; set; }

    // Dados próprios do fornecedor
    public string NomeEmpresa { get; set; } = "";
    public string Estado { get; set; } = "Pendente"; // Pendente / Aprovado

    // Produtos deste fornecedor
    public ICollection<Produtos>? Produtos { get; set; }
}
```

### 4.2.4. Produtos

Entidade central que representa os produtos disponíveis para venda:

```
// GestaoLoja/Entities/Produtos.cs
public class Produtos
{
    public int Id { get; set; }
    public string Nome { get; set; } = string.Empty;

    // Preços conforme enunciado
}
```

```

        public decimal PrecoBase { get; set; } // Definido pelo fornecedor
        public decimal Percentagem { get; set; } // Definida pelo
admin/funcionário
        public decimal PrecoFinal { get; set; } // PrecoBase + (PrecoBase *
Percentagem / 100)

        public string Estado { get; set; } = "Pendente"; // Pendente / Ativo /
Inativo
        public int Stock { get; set; }
        public string Imagem { get; set; } = "semfoto.png";

        // Relações
        public int CategoriaId { get; set; }
        public Categorias? Categoria { get; set; }

        public int? SubcategoriaId { get; set; } // Opcional
        public Subcategoria? Subcategoria { get; set; }

        public int ModoEntregaId { get; set; }
        public ModoEntrega? ModoEntrega { get; set; }

        public int FornecedorId { get; set; }
        public Fornecedor? Fornecedor { get; set; }
    }

```

#### 4.2.5. Categorias e Subcategorias

```

// GestaoLoja/Entities/Categorias.cs
public class Categorias
{
    public int Id { get; set; }
    public string Nome { get; set; } = string.Empty;
    public string? Imagem { get; set; }

    public ICollection<Subcategoria>? Subcategorias { get; set; }
    public ICollection<Produtos>? Produtos { get; set; }
}

// GestaoLoja/Entities/Subcategoria.cs
public class Subcategoria
{
    public int Id { get; set; }
    public string Nome { get; set; } = string.Empty;
    public string? Imagem { get; set; }

    public int CategoriaId { get; set; }
    public Categorias? Categoria { get; set; }

    public ICollection<Produtos>? Produtos { get; set; }
}

```

#### 4.2.6. Vendas e LinhasVenda

```

// GestaoLoja/Entities/Vendas.cs
public class Vendas

```

```

{
    public int Id { get; set; }
    public string Data { get; set; } = DateTime.Now.ToString("yyyy-MM-dd
HH:mm:ss");
    public string Estado { get; set; } = "Pendente"; // Pendente /
Confirmada / Rejeitada / Expedida

    public int ClienteId { get; set; }
    public Cliente? Cliente { get; set; }

    public ICollection<LinhasVenda>? LinhasVenda { get; set; }
}

// GestaoLoja/Entities/LinhasVenda.cs
public class LinhasVenda
{
    public int Id { get; set; }

    public int? VendaId { get; set; }
    public Vendas? Venda { get; set; }

    public int? ProdutoId { get; set; }
    public Produtos? Produto { get; set; }

    public int Quantidade { get; set; }
    public decimal Preco { get; set; } // Preço no momento da compra
}

```

## 5. Regras de Negócio

O núcleo da plataforma **MyMEDIA** é regido por um conjunto estrito de regras de negócio que garantem a integridade dos dados, a segurança das transações e o correto funcionamento do modelo de intermediação comercial. As principais regras implementadas são descritas detalhadamente abaixo:

### 5.1. Regras Gerais

- A plataforma MyMEDIA é intermediária entre fornecedores e clientes.
- A plataforma cobra uma percentagem sobre cada venda, definida pelo administrador/funcionário.
- O preço final é calculado como:  $\text{PrecoFinal} = \text{PrecoBase} + (\text{PrecoBase} \times \text{Percentagem} / 100)$ .
- Existem 4 perfis de utilizador: Administrador, Funcionário, Cliente e Fornecedor.

### 5.2. Regras Utilizadores

- Novos registos de Clientes ficam com estado "Pendente" até aprovação por Admin/Funcionário.
- Novos registos de Fornecedores ficam com estado "Pendente" até aprovação por Admin/Funcionário.
- Utilizadores com estado "Pendente" não podem fazer login.
- Utilizadores com estado "Inativo" não podem fazer login.
- Apenas Administradores podem gerir Funcionários.
- Administradores e Funcionários podem gerir Clientes e Fornecedores.
- Um utilizador não pode desativar o seu próprio registo.

### 5.3. Regras de Produtos

- Produtos inseridos por Fornecedores ficam com estado "Pendente".
- Produtos editados por Fornecedores voltam ao estado "Pendente" para reaprovação.
- Apenas produtos com estado "Ativo" são visíveis para os clientes.
- Apenas Admin/Funcionário podem alterar o estado de produtos para "Ativo".
- A percentagem de lucro é definida pelo Admin/Funcionário na aprovação do produto.
- Produtos não podem ser apagados se existirem vendas associadas.
- Fornecedores apenas podem gerir os seus próprios produtos.

#### 5.4. Regras de Vendas

- Apenas utilizadores autenticados como Cliente podem finalizar compras.
- Uma venda inicia com estado "Pendente".
- Admin/Funcionário podem Confirmar ou Rejeitar vendas pendentes.
- Ao confirmar uma venda, o stock dos produtos é automaticamente decrementado.
- Vendas confirmadas podem ser marcadas como "Expedida".
- O cliente pode simular o pagamento de vendas pendentes.
- Fornecedores podem ver o histórico de vendas dos seus produtos.

#### 5.5. Regras de Categorias e Subcategorias

- Uma categoria pode ter múltiplas subcategorias.
- Um produto pertence obrigatoriamente a uma categoria.
- Um produto pode opcionalmente pertencer a uma subcategoria.
- Categorias não podem ser apagadas se tiverem produtos associados.
- Subcategorias não podem ser apagadas se tiverem produtos associados.

#### 5.6. Regras de Stock

- Produtos com stock = 0 aparecem como "Esgotado".
- Produtos esgotados não podem ser adicionados ao carrinho.
- O stock é atualizado automaticamente quando uma venda é confirmada.

## 6. Credenciais de Acesso

### 6.1. Aplicação de Gestão (GestãoLoja)

| Role          | Email           | Password  |
|---------------|-----------------|-----------|
| Administrador | admin@gestao.pt | Admin123! |
| Funcionário   | func@gestao.pt  | Func123!  |

### 6.2. Aplicação Frontend (MyMediaWeb / MyMediaMAUI)

| Role       | Email                | Password     |
|------------|----------------------|--------------|
| Cliente    | cliente@teste.pt     | Cliente123!  |
| Cliente    | cliente2@teste.pt    | Cliente2123! |
| Cliente    | cliente3@teste.pt    | Cliente3123! |
| Fornecedor | fornecedor@teste.pt  | Forn123!     |
| Fornecedor | fornecedor2@teste.pt | Forn2123!    |
| Fornecedor | Fornecedor3@teste.pt | Forn3123!    |

## 7. Conclusão

O trabalho prático foi concluído com sucesso, cumprindo todos os requisitos especificados no enunciado. A plataforma MyMEDIA representa uma solução completa e funcional para comércio eletrónico de produtos multimédia.

Objetivos alcançados:

- Três aplicações desenvolvidas e funcionais (Frontend Web/MAUI, API, Backoffice)
- Código único partilhado entre Web e MAUI
- CSS Framework personalizado sem dependências externas
- Todas as funcionalidades implementadas
- Quatro perfis de utilizador com permissões diferenciadas
- Regras de negócio implementadas e respeitadas
- O desenvolvimento deste projeto permitiu consolidar conhecimentos em:
- .NET 8 e C# 12
- Blazor Web App e Blazor Hybrid (MAUI)
- ASP.NET Core Web API
- Entity Framework Core
- ASP.NET Core Identity e JWT
- Desenvolvimento de CSS personalizado
- Arquitetura de software distribuída